



Year 2 Project

Balance car. Hamster Ball

Name. Qi Cheng	ID. 201768864
Name. Shaotong Hong	ID. 201769037
Name. Zijin Wu	ID. 201769693
Name. Yunqi Wang	ID. 201769616
Name. Junyang Xiao	ID. 201769708

Supervised by. Dr. Heba Lakany

Department of Electrical Engineering and Electronics

15 March, 2024

Abstract

This project introduces an autonomous spherical vehicle called the "Hamster Ball," which has a PID control system to increase stability and manoeuvrability. The paper highlights the need for better personal robotic transportation and explains the incorporation of complex control methods. The results show that the PID system is highly accurate and adaptable to environmental factors, leading to precise navigation. The "Hamster Ball" is considered a flexible tool with possible applications ranging from recreational to educational fields. Future improvements will involve advanced features to enhance autonomous functions and adaptability in various settings.

Declaration

I confirm that I have read and understood the University's definitions of plagiarism and collusion from the Code of Practice on Assessment. I confirm that I have neither committed plagiarism in the completion of this work nor have I colluded with any other party in the preparation and production of this work. The work presented here is my own and in my own words except where I have clearly indicated and acknowledged that I have quoted or used figures from published or unpublished sources (including the web). I understand the consequences of engaging in plagiarism and collusion as described in the Code of Practice on

Content

Introduction	7
Materials and Methods	9
2.1 Materials List	9
Mechanical Components	9
Hardware	9
Software	9
2.2 Kinematics Analysis	10
2.3 Mechanical Structure Design	13
2.3.1 Testing Platform	13
2.3.2 Final Product	15
2.4 Hardware and Circuit Design	20
2.5 PID Analysis	22
2.5.1 Design of Velocity Loop	23
2.5.2 Design of Balance Loop	27
2.6 Program and code design	28
2.6.1 Kinematic Control Algorithms	29
2.6.2 Sensor Reading	32
2.6.3 Coding Implementation	35
Results	37
3.1 General performance	37
3.2 Kinematics Analysis	37
3.3 Mechanical Structure Design	37
3.3.1 Testing Platform	37
3.3.2 Final Product	38
3.4 Hardware and Circuit Design	40
3.5 Program and Code Design	41
3.5.1 Control Performance	41
3.5.2 Unexpected System Dead Halt	45
3.5.3 Final Representation	47
Discussion and Conclusion	48
4.1 Revisiting Previous Sections	48
4.2 Limitations Current and Future Work Applications	48
4.3 Achievement and Contribution	48
4.4 Reflection	49
References	50
Appendix	51
A.1 Y2 project (ELEC222/ELEC273) - Contribution to project deliverables	51
A.2 Y2 project (ELEC222/ELEC273) – <i>Role allocation (responsibility matrix)</i>	52
A.3 Code	53

List of figures

Figure 1. Inspirations	7
Figure 2. Validation design	10
Figure 3. Schematic Diagram of Movement	11
Figure 4. Interfering part with the wheel	14
Figure 5. Early testing platform with hardware	14
Figure 6. Chassis reinforcement parts	15
Figure 7. Motor placement	16
Figure 8. Drive system distribution when $\theta=90^\circ$	17
Figure 9. Comparison of design phases	17
Figure 10. Insulating gasket	18
Figure 11. Chassis detail	18
Figure 12. The underside of the chassis	19
Figure 13. 3D printing method	19
Figure 14. Connection of hardware modules (without voltage reduction module).....	20
Figure 15. Operating Principle of Velocity Loop Control	23
Figure 16. Response Under P Control	25
Figure 17. Response Under PI Control for velocity loop	26
Figure 18. Operating Principle of Balance Loop Control	27
Figure 19. Design sectors for 'Hamster Ball'	29
Figure 20. General structure of control system	36
Figure 21. Comparison of beacon bases before and after the adjustment.....	38
Figure 22. Self-balancing (while power off).....	39
Figure 23. Machanical design where need to improve	39
Figure 24. Operation problems	41
Figure 25. Final code structure	47

List of Tables

Table 1. Contribution to project deliverables.....	51
Table 2. Role allocation	52
Table 3. Responsibility Matrix	52

List of code listings

Listing 1. Inverse kinematic	29
Listing 2. Forward kinematic.....	30
Listing 3. Close loop PD control in Roll direction	30
Listing 4. Close loop PD control in Pitch direction	31
Listing 5. Close loop PI control in X and Y direction	31
Listing 6. Lines of reading MPU DMP	32
Listing 7. Encoder reading method	33
Listing 8. PS2 Reading	34
Listing 9. Botton detection & variable adjustment	34
Listing 10. Variable display	35
Listing 11. PWM limiting	43
Listing 12. Integrate limitation.....	44
Listing 13. Fast decrease in integration	45
Listing 14. Simple low-pass filter	45

Introduction

Background and Problem Statement

The field of self-driving cars has always looked for ways to improve productivity and security [1]. In answer to this quest, "Hamster Ball," a self-balancing autonomous vehicle, is born, using PID control theory as its foundation. Several science fiction concepts, such as the tourist ball in Jurassic World (Figure 1 b) and BB8 in Star Wars (Figure 1 a), served as inspiration for this design.



a) BB8 in Star Wars



b) Tourist ball in Jurassic World

Figure 1. Inspirations

Even with all of the advancements in autonomous vehicles, integrating self-stabilization techniques is still a difficult frontier, especially in small sizes.

Terminology and Field Importance

PID control, or proportional, integral, and derivative control, is a basic automation concept that is essential for improving how vehicles react to changing conditions [2]. This project makes a substantial contribution to the field of robotics and automated systems by highlighting the vehicle's agility and dependability. It is believed that even in uncertain environments, a finely tuned PID system can greatly improve a tiny autonomous vehicle's ability to balance and navigate.

3D printing, Engineering has been transformed by 3D printing because it makes it possible to create complex geometries, create unique components, and prototype quickly—things that traditional manufacturing is not able to do. It

drastically shortens the time and expense from design to manufacturing, greatly expedites the design process, and enables quick idea testing.

Research Landscape

Most existing research concentrates on large-scale automobile applications, frequently ignoring the market for more compact, adaptable autonomous devices such as the "Hamster Ball." This project optimises and miniaturises PID control for a new application, building on previous research.

Research Gap and Objectives

The practical application of PID controllers in small, autonomous vehicles that can traverse challenging terrains is a glaring research void. The goal of "Hamster Ball" is to close this gap by showcasing a reliable, flexible, and approachable model for leisure and personal transportation.

Report Overview

This report delves into the analysis and design of the PID control system tailored for the "Hamster Ball," evaluates its performance, and discusses the implications of its deployment in real-world scenarios. Sections II and III will articulate the design process and the results of various tests conducted to fine-tune the system, while Section IV will be the conclusion and discussion about the group project

Materials and Methods

2.1 Materials List

Mechanical Components

Acrylic Spherical shell (diameter 250mm)
eSUN PLA+ 3D printing filament 1kg
TINMORRY PETG 3D printing filament 1kg
Gueenky Threaded Knurled Inserts M3*4
Zxglxinyao Hex Lock Nuts M2; M3
Hakki Bolts M2*6; M3*8; M3*16
Mgichoom 243 Screw Sealant Threadlocker

Hardware

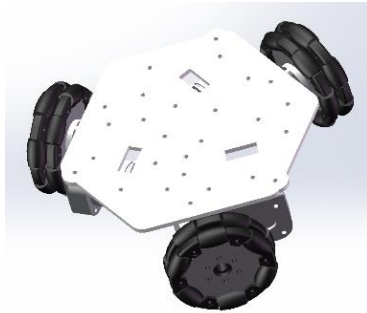
Arduino MEGA 2560 R3
D24A Motor drive(Integrates two TB6612s and a voltage regulator)
WHEELTEC MG513X DC Reduction Motor with Hall encoder(1.28)
ACE 3S Battery 11.2V 20C 2000mah
0.96 inch 128*64 IIC OLED Screen
MPU 6050
PS2 controller and receiver

Software

SOLIDWORKS 2021
Bambu Studio
Arduino IDE 2.2.1
Fritzing

2.2 Kinematics Analysis

After finalizing the design concept for the project, the balancing vehicle takes on a form similar to that of a hamster ball, consisting of an acrylic sphere shell and a drive platform. The ultimate goal is to achieve omnidirectional movement. The drive platform comprises three omnidirectional wheels, as illustrated in (Figure 2 a)



a) Technical Validation Chassis



b) Omnidirectional wheels

Figure 2. Validation design

Each omnidirectional wheel consists of a hub and numerous small rollers fixed around the periphery, with a 90° angle between the wheel axis and the roller axis, as depicted in (Figure 2 b). Consequently, each wheel can rotate about its axis or move along the direction of the rollers' axes, perpendicular to the ground contact, pivoting about the contact point between the wheel and the ground. The platform within the sphere mainly utilizes the first two modes of motion.

To facilitate control over the forward/backward, left/right, and rotational movements of the hamster ball, as well as to obtain feedback for adjusting the operational state based on the actual speeds of each wheel, it is necessary to perform essential kinematics analysis. This includes both forward and inverse kinematics, to respectively map the rotational speeds of the three wheels to the velocities along the X, Y, and Z axes (i.e., platform rotation speed), and to map the velocities along the X, Y, and Z axes to the rotational speeds of the three wheels, as illustrated in (Figure 3).

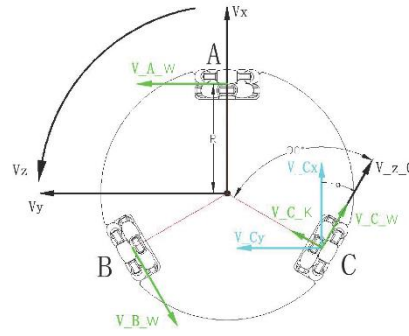


Figure 3. Schematic Diagram of Movement

R . The rotation radius, the distance from the Omni-wheel to the center of the robot, in meters (m).

V_x . The robot's forward and backward movement speed.

V_y . The robot's left and right movement speed.

V_z . The robot's rotational speed around center of the robot.

V_{A_w} , V_{B_w} , V_{C_w} . The linear speed of omni-wheels A, B, C, generated by the rotation of the motor driving the omni-wheel hub, with counterclockwise around shaft being positive.

The following discusses parameters relevant to omni-directional wheel C, as depicted in Fig. 3. Only focus on omni-directional wheel C below, and similar parameters apply to the other two omni-directional wheels, A and B.

V_{C_K} . The linear speed of the roller in contact with the ground of the omnidirectional wheel C.

V_{z_c} . The linear velocity of the center rotating around point O.

V_{C_x} . The forward and backward motion velocity of the centroid of the omnidirectional wheel C.

V_{C_y} . The left and right movement velocity of the centroid of the omnidirectional wheel C.

α . The angle between the forward direction of the omnidirectional wheel C ($\alpha=30^\circ$).

First, the relationship between the front and back moving speeds V_{c_x} , V_{c_y} and the overall moving speeds V_x , V_y and V_z around point O of the omnidirectional wheel C centroid is obtained. The car body and the centroid of the three omnidirectional wheels can be considered as a rigid body, then the velocity decomposition can be obtained as follows.

$$V_{c_x} = V_x + V_z * R * \cos \alpha \quad \text{Eq.1}$$

$$V_{c_y} = V_y + V_z * R * \sin \alpha \quad \text{Eq.2}$$

The relationship between the omnidirectional wheel C centroid velocity and the overall robot velocity, and our omnidirectional wheel C centroid velocity is fundamentally generated by the combination of V_{c_w} and V_{c_k} . Through velocity decomposition.

$$V_{c_x} = V_{c_k} * \sin \alpha + V_{c_w} * \cos \alpha \quad \text{Eq.3}$$

$$V_{c_y} = V_{c_k} * \cos \alpha - V_{c_w} * \sin \alpha \quad \text{Eq.4}$$

Combining Eq.1, Eq.3 and Eq.2, Eq.4 yields the following Eq.5 and Eq.6

$$V_{c_k} * \sin \alpha + V_{c_w} * \cos \alpha = V_x + V_z * R * \cos \alpha \quad \text{Eq.5}$$

$$V_{c_k} * \cos \alpha - V_{c_w} * \sin \alpha = V_y + V_z * R * \sin \alpha \quad \text{Eq.6}$$

Where Eq.6 can be converted to.

$$V_{c_k} = \frac{V_y - V_z * R * \sin \alpha + V_{c_w} * \sin \alpha}{\cos \alpha} \quad \text{Eq.7}$$

Combining Eq.7 with Eq.5 gives.

$$\tan \alpha (V_y - V_z * R * \sin \alpha + V_{c_w} * \sin \alpha) + V_{c_w} * \cos \alpha = V_x + V_z * R * \cos \alpha \quad \text{Eq.8}$$

It can be seen from Fig.3 that $\alpha = 30^\circ$, so the inverse kinematics analysis of C round can be obtained after simplification.

$$V_c = \frac{\sqrt{3}}{2} v_x - \frac{1}{2} v_y + v_z \cdot R \quad \text{Eq.9}$$

A similar method can be used to calculate the relationship between round A, B, C and V_x , V_y , V_z . Finally, the inverse kinematics can be solved as follows.

$$V_A = v_y + v_z \cdot R \quad \text{Eq.10}$$

$$V_B = -\frac{\sqrt{3}}{2} v_x - \frac{1}{2} v_y + v_z \cdot R \quad \text{Eq.11}$$

$$V_c = \frac{\sqrt{3}}{2} v_x - \frac{1}{2} v_y + v_z \cdot R \quad \text{Eq.12}$$

In the same way which can also get the results of the forward solution of the kinematics.

$$v_x = \frac{V_C - V_B}{\sqrt{3}} \quad \text{Eq.13}$$

$$v_y = \frac{2V_A - V_B - V_C}{3} \quad \text{Eq.14}$$

$$v_z = \frac{V_A + V_B + V_C}{3R} \quad \text{Eq.15}$$

2.3 Mechanical Structure Design

In the development of the mechanical components for the hamster ball, SolidWorks was primarily utilized for modelling and designing the basic structure. Through the construction of models, it facilitated the rapid generation of envisioned appearances, enhancing efficiency for real-world installation and debugging. Simultaneously, the design of the overall structure prioritized mechanical performance while considering overall coordination and aesthetics.

The design objectives aimed to ensure non-interference among hardware installations while maintaining simplicity and environmental friendliness. Furthermore, it ensured the center of mass was centrally balanced in a stationary state. During propulsion, efficient contact with the ball shell was ensured. Post-assembly, the structure maintained stability with a certain degree of rigidity and ease of operation.

2.3.1 Testing Platform

The focus of hamster ball development primarily centered around the drive platform, with the ball shell being procured as a customized acrylic shell. Initially, the team wanted to directly design and 3D print the final platform. However, upon careful consideration, team members realized that continually inputting code through computer connections to the platform and then installing and debugging within the shell was highly inefficient and inconvenient. Thus, they

pivoted to first developing a simplified version of the prototype chassis, conducting initial testing and debugging on flat ground rather than within the ball.

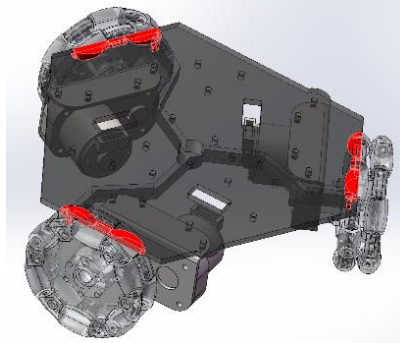


Figure 4. Interfering part with the wheel

During the development and testing of the chassis, the goal was to construct a simple, efficient platform capable of accommodating hardware devices without conflicts. This principle was upheld during design in SolidWorks, where a draft was finalized in just 30 minutes and immediately 3D printed. Due to a measurement oversight, a conflict emerged during wheel installation, as shown in (Figure 4), with the red area indicating interference.

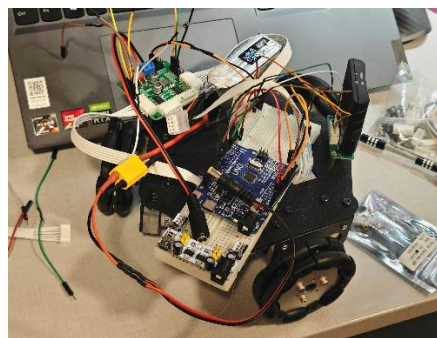


Figure 5. Early testing platform with hardware

To address the issue, team members used SolidWorks to cut certain parts inward from the three sides of the original model's chassis that connect to the drive motors, while preserving the rest of the platform's original design. This allowed the test platform to meet the most basic and practical testing effects within a shorter time frame. Moreover, after quickly and easily installing the hardware that needed debugging as shown in (Figure 5), the team members began preliminary testing and adjustment work.

Key Structural and Component Features:

To ensure the platform could withstand the installation of relatively large component masses and resist the torque generated during movement to avoid chassis deformation resulting in poor contact between wheels and the ground, a structure with three reinforcing beams depicted in blue in (Figure 6) was designed to stabilize the structure. Additionally, to facilitate firmware fixation and installation, a certain number of 3mm screw holes were reserved, providing convenience for subsequent debugging, especially with intricate loads of interconnected circuitry on each hardware board.

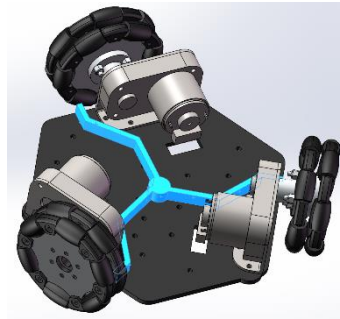


Figure 6. Chassis reinforcement parts

According to mechanical principles, the number of actuating parts of a mechanism should be equal to its degrees of freedom. Therefore, to control three degrees of freedom, three independent inputs are required. Each omni-directional wheel can be regarded as an actuating part. Hence, to achieve control over three planar degrees of freedom, at least three omni-directional wheels are necessary. Thus, with a platform composed of three such wheels, omnidirectional movement can be realized.

2.3.2 Final Product

The Selection of Motor and Gearbox

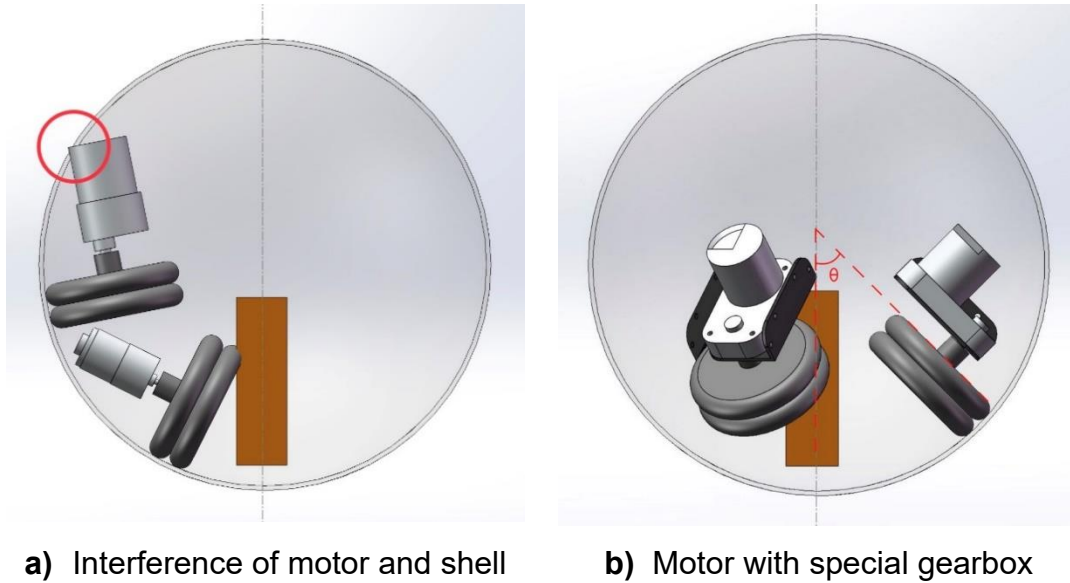


Figure 7. Motor placement

Prior to the architectural development of the vehicle, it is imperative to select the fundamental components—the motor and the gearbox. Due to the limited space inside the sphere and the need to ensure good contact between the wheels and the spherical shell, there are inevitable limitations to the utilization of conventional motors. motor would interfere with the spherical shell or its size that are too small to provide enough power, as shown in (Figure 7 a). Consequently, for this project, the motor equipped with a specialized gearbox as shown in (Figure 7 b) has been chosen to circumvent these limitations.

Symmetrical and Low Center of Gravity Design

In addition to utilizing a PID controller, the distribution of the center of gravity is crucial to the balance performance of a self-balancing vehicle. The objective of the mechanical design is to develop a vehicle that can self-balance even without a PID controller as a tumbler. To achieve this objective, it is necessary to position the vehicle's center of gravity as low as possible and proximate to its central axis. Since the main weight of the vehicle is concentrated in the motor and battery, it is essential to design these heavy components with symmetry relative to the vehicle's central axis and to locate them as low as possible.

The Angle Between The Central Axis and The Wheel

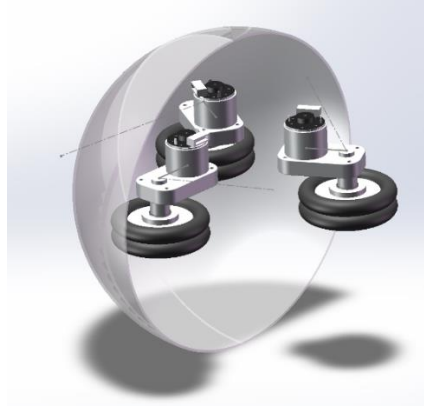


Figure 8. Drive system distribution when $\theta=90^\circ$

The vehicle is propelled indirectly through the motion of the spherical shell rather than being directly driven by wheels. Ensuring high driving efficiency in such a mechanism is of paramount importance.

Same as the testing platform, when the angle (θ) equal to 0° , the driving efficiency reaches its peak(100%). As shown in (Figure 8), when the angle (θ) equals equal to 90° , the driving efficiency is 0%, irrespective of how the three wheels are actuated, motion cannot synthesize a horizontal velocity, resulting only in rotation. Therefore, minimizing the angle between the wheels and the central axis could mitigate the adverse effects of this driving mode on driving efficiency.

The Design of Chassis

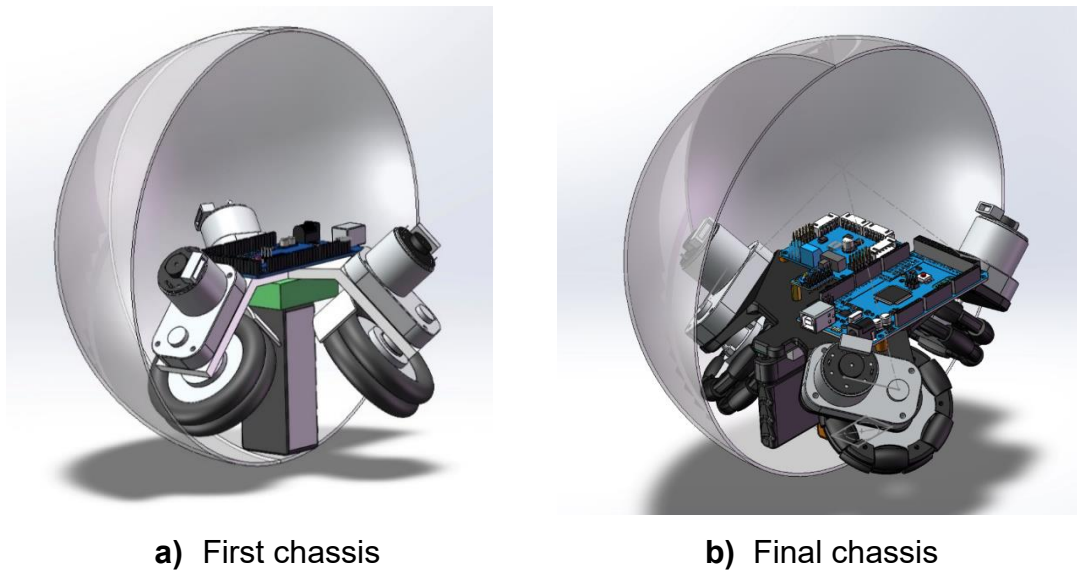


Figure 9. Comparison of design phases

Based on the above requirements, the first chassis was designed in (Figure 9

a). But there was a serious problem. there was no enough space for all the hardware modules, and the center of gravity wasn't low enough.

To solve this problem, the motor and gearbox was rotated around its axis by a certain angle as shown in (Figure 9 b). after rotating the motor, there is enough place for the hardware to place, and the center of gravity of the vehicle is lowered further

Hardware Placement

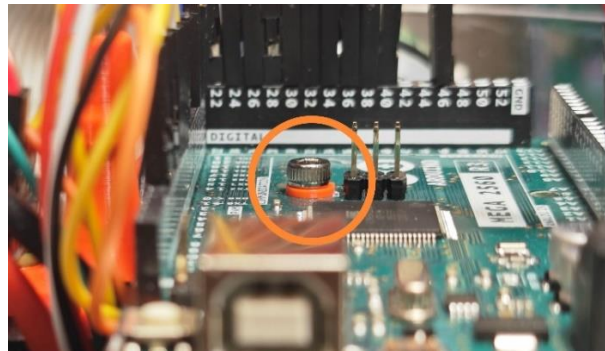


Figure 10. Insulating gasket

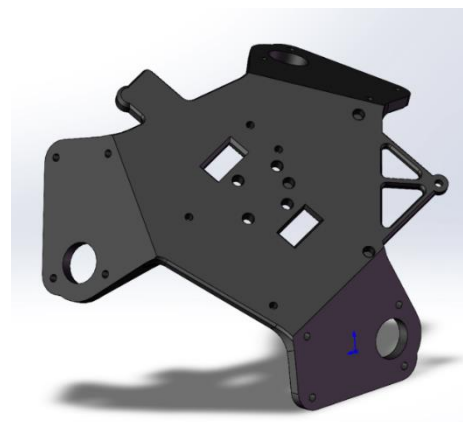
Same as the motor and battery, Hardware modules also need to be evenly placed on the frame as (Figure 9 b) to ensure that the vehicle's centre of gravity is roughly located on the central axis.

When securing hardware modules, it is necessary to place an insulating gasket (fabricated using 3D printing in this project) between the bolts and the circuit board shown as in (Figure 10) to prevent the potential risk of short-circuiting.

Ensure The Strength and Rigidity of The Chassis



a) The underside of the chassis



b) The upper surface of the chassis

Figure 11. Chassis detail

Compared to the testing platform, the thickness of the chassis has been increased, and reinforcement ribs have been designed on it, as shown in (Figure 11), to prevent deformation. This ensures optimal contact between the wheel and the spherical shell, thereby maximizing the avoidance of slippage and inefficient driving phenomenon.

Beacon

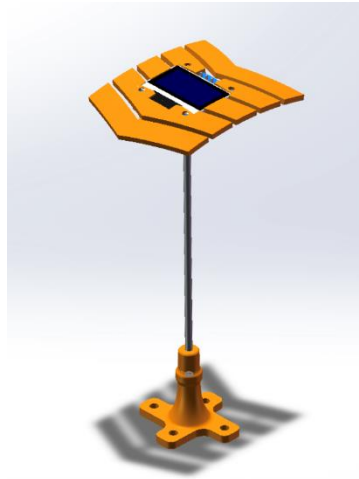
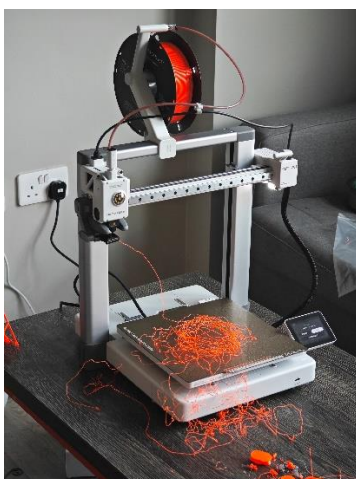


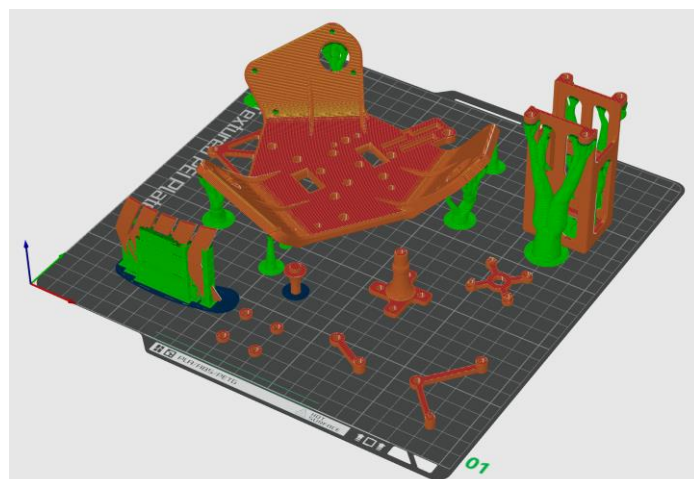
Figure 12. The underside of the chassis

Because of the symmetrical design, it is hard to distinguish the direction during operation. So, the light weight beacon with a screen as (Figure 12) was designed to solve this problem.

Limitation of 3D Printing



a) Failed 3d printing



b) Slicing results of 3D printed components

Figure 13. 3D printing method

The structural components are primarily manufactured using 3D printing, and

the limitations of 3D printing need to be considered (Figure 13 a). For example, it is necessary to ensure that there are no protrusions above the chassis, as illustrated in (Figure 13 b) to prevent overhang printing. Overhang printing can reduce the strength and precision of the structure and may even lead to failures in 3D printing (Figure 13 a).

2.4 Hardware and Circuit Design

In this project, to expedite product development within a limited time, the finished hardware modules were widely used, and these modules were connected through soldering or Dupont wires to enhance development efficiency while avoiding potential issues associated with PCB design and manufacturing. The main aims of the hardware and circuit design was to methodically connect all modules, preventing interference among different communication methods and successfully driving all modules. The following sections provide detailed explanations of the basic functions, characteristics, and reasons for selecting these components over other competitive products.

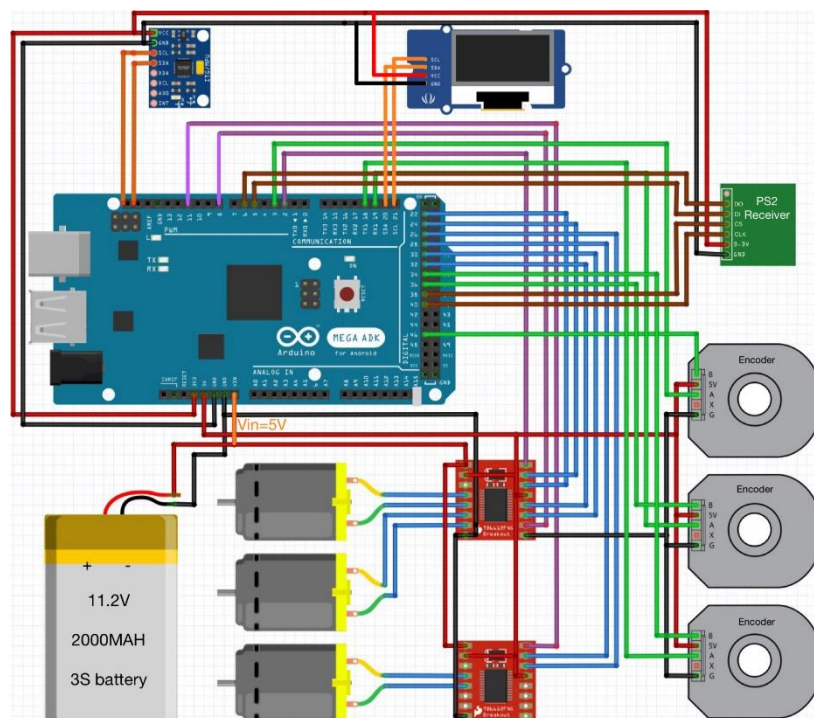


Figure 14. Connection of hardware (without voltage reduction module)

Microprocessor (Arduino MEGA 2560 R3)

The project utilizes an Arduino MEGA 2560 R3 board as the microprocessor, enabling the execution of complex control algorithms and ensuring the vehicle's operations. It is equipped with an I2C bus and ample interfaces, providing support for the connection and communication of numerous hardware modules. The reason why not using other more powerful microprocessors, such as Raspberry Pi or STM32, is because the main objective of this project is to apply and explore PID controllers within a limited development timeframe(4 weeks). The capabilities of the Arduino are sufficient for the completion of this project, and its simplicity of operation allows for the avoidance of complex embedded development time.

Motor Drive (Integrates two TB6612s and a voltage reduction module)

In this project, three brushed DC motors are utilized and it is necessary to reduce the voltage supplied by the battery from 11.2V to 5V and 3.3V in order to power other hardware modules. Therefore, a motor driver module that includes two TB6612 chips (each capable of driving two DC motors) and a voltage reduction module has been selected.

Motor with Hall Encoder (WHEELTEC MG513X)

This project has selected the WHEELTEC MG513X brushed DC motor with the gear ratio of 1.28, which can provide sufficient torque for this vehicle's unique drive method while being stable and durable. Additionally, it is equipped with a Hall encoder, allowing the PID controller to read the motor's actual speed and direction.

Battery

After research, it was determined that the stall current of the motors used in this project is 3A, with a rated voltage of 11.2V. Additionally, considering the need to power other hardware devices, the battery must have sufficient capacity and a discharge capability exceeding 10A, which is challenging for ordinary dry cells

to achieve. Given the limited space inside the sphere, the battery's size cannot be too large. Therefore, a 3S 2000mAh 20C model aircraft battery was selected for this project.

$$I_{MAX} = 2000mah \times 20c = 20A > 10A$$

According to the formula, this battery meets the project's requirement

OLED Screen

The communication between the OLED screen and the microprocessor is facilitated through the I2C bus, with its SDA and SCL pins connected to the corresponding ports on the Arduino (either labeled as SDA & SCL or ports 20 & 21). This screen is utilized to display PID and speed parameters, thereby enabling convenient wireless adjustments.

MPU 6050

The MPU6050 communicates with the microprocessor via the I2C bus, with its SDA and SCL pins connected to the I2C bus alongside the OLED screen. It is equipped with a 6-axis gyroscope and accelerometer module, used by the PID controller to read vehicle attitude information, including acceleration and angular velocity along the Yaw, Pitch, and Roll axes.

2.5 PID Analysis

In this project, PID control theory is employed as the core of the vehicle's control system. After identifying the project's focus on a balancing vehicle, extensive analysis and research on PID systems were completed. The essence of PID control lies in the analysis and control of errors to achieve and maintain system stability at a target value [3]. By performing proportional, integral, and differential operations on the errors and summing them up, the control signal is generated. This signal is converted into a PWM signal and re-entered into the motor control system. In this way, the error in the motor speed output will be

reduced. Based on the foregoing analysis, it is logical to design the vehicle's PID control system to operate within a continuous loop, iterating repeatedly as the vehicle progresses [4]. With high-frequency correction, the error in motor output speed will be significantly reduced, making the vehicle more stable during movement.

The concept of superposition is applied in the vehicle's PID control system. Considering the need for the vehicle to maintain balance while moving, the PID control system is divided into two independent loops. a velocity loop and a balance loop. The velocity loop addresses the system's stable speed during the vehicle's movement, while the balance loop aims to keep the vehicle stable at a mechanical 0-degree position. The balance loop and velocity loop are parallel in the code. Through this design, within each main loop of the Arduino code, the PID control system acquires two correction values for the motors. one derived from the velocity loop and the other from the balance loop. The sum of these two correction values serves as the total control signal of the PID control in each loop iteration, input into the motors. The specific logic of the PID system is designed through this method of decomposition and superposition.

2.5.1 Design of Velocity Loop

Based on the analysis in the previous section, the velocity loop of the vehicle has been designed as depicted in the following flowchart (Figure 15).

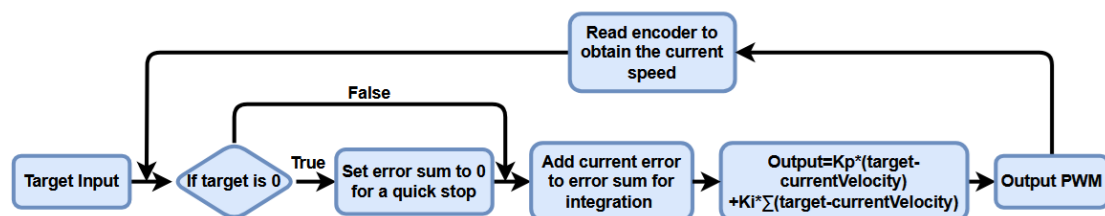


Figure 15. Operating Principle of Velocity Loop Control

As illustrated in (Figure 15), the system initially reads a speed value as the target, which is obtained through a PS2 controller and decoded into a velocity

value. Subsequently, the system evaluates whether the target value is zero. If the target value is zero, the system will directly output a PWM signal with a value of zero to the motors, thereby causing the motors to stop quickly. If the target value is not zero, the process skips zeroing and proceeds to error calculation. At this stage, the system reads an actual speed value from the motor encoder and subtracts it from the target value to obtain the instantaneous error value. This instantaneous error value is accumulated and stored in a variable as the sum of error, serving as the integral of the error. The error equation is discrete because the recorded error is calculated from the values read by the encoder at each instance. Since the integral of a discrete equation involves accumulation, the summation is used to compute the error's integral. Then, the instantaneous error value is multiplied by a parameter K_p , and the total error sum is multiplied by a parameter K_i . The sum of these two products forms the velocity value of the control signal, also known as the correction value. Finally, this corrected velocity value is decoded into a PWM signal input to the motors to complete PID control and enter the next cycle.

In this cycle, the step of determining whether the target value is zero is included to ensure the vehicle can stop quickly when the user ceases to operate the PS2 controller. Without this conditional statement, the system would continue to calculate errors and perform subsequent speed correction calculations. Since the proportional parameter K_p and the integral parameter K_i are relatively small, the value of the speed correction would also be small. This would cause the vehicle to slowly adjust its speed until it reaches zero. For instance, in the case of a vehicle moving at high speed, if the target value is set to zero, the vehicle's speed would decrease slowly instead of stopping abruptly. This significantly increases the vehicle's braking time, making operation difficult. This is the rationale behind the inclusion of the conditional statement.

In the initial design, the calculation of the correction value in the velocity loop

did not include integral correction, only proportional correction. This initial design was based on the principle of achieving complex control with the simplest design. However, after more than ten rounds of testing, the idea of using proportional control alone was rejected. As shown in (Figure 16) below, with proportional control only, no matter how the proportional parameter K_p was adjusted, it was impossible to eliminate the steady-state error.

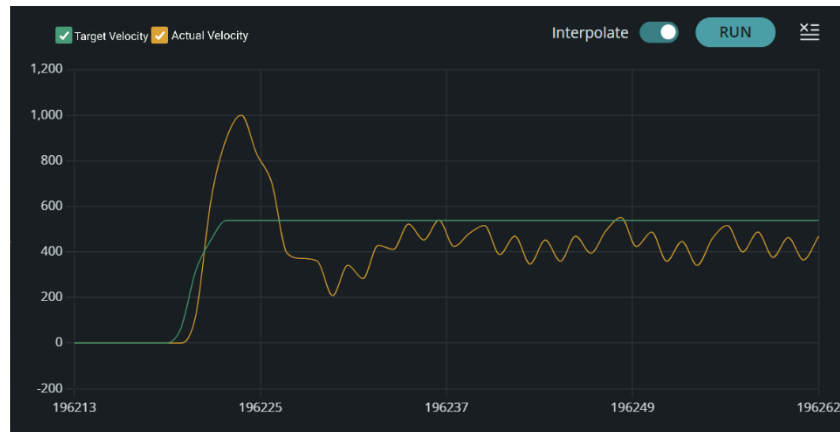


Figure 16. Response Under P Control

As depicted in (Figure 16), at steady state, there remains a discrepancy between the actual value and the target value. This discrepancy is caused by the steady-state error. To address the steady-state error, integral control was introduced into the calculation of the correction value in subsequent design considerations, as shown in (Figure 15).

To eliminate the steady-state error, it is necessary to incorporate an integral component in the calculation of the velocity correction value [5]. Through the combination of proportional control and integral control, the system response is optimized for quickness and can eliminate static error once the system stabilizes. Over time, the integral of the error increases. Thus, even if the instantaneous error is small, the integral component of the error increases over time, driving an increase in the controller's output to further reduce the steady-state error until it equals zero.

Why Use PI Control for Velocity Loop

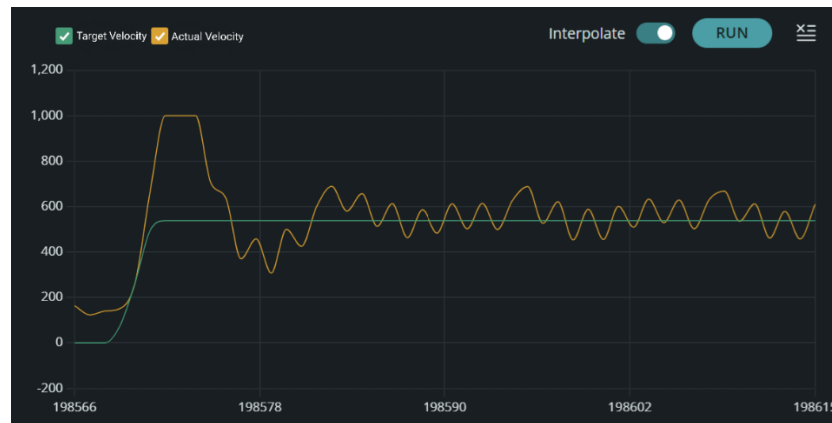


Figure 17. Response Under PI Control for velocity loop

Due to the complexity of the vehicle's mechanical structure, there is an error in the mechanical midpoint (balance point) of the vehicle. Without a velocity loop to control this, the vehicle might continuously move forward or backwards to maintain this erroneous mechanical midpoint. Hence, the inclusion of a velocity loop aims to correct the vehicle's angle, thereby stabilizing the vehicle at the target speed, as illustrated in (Figure 17). When the vehicle persistently moves forward, the velocity loop accelerates the vehicle, causing it to lean back, at which point the balance control will cause the vehicle to move backward to maintain balance. Moreover, PI control ensures a constant speed even in the face of load changes or external disturbances. Precision in speed control is paramount, but the reason for not employing differential control in the velocity loop is due to the relatively slow dynamic changes in speed control, where the issues of steady-state overshoot and oscillations are not as pronounced as in balance control. As shown in (Figure 16), the vehicle can utilize the increased torque at startup for rapid acceleration due to proportional control. Differential control would improve the system's transient response and reduce overshoot. Therefore, in the design phase, differential control was not used in the calculation of the corrected speed; instead, a combination of proportional and integral control was utilized.

2.5.2 Design of Balance Loop

The balance loop of the vehicle has been designed as shown in the following (Figure 18).

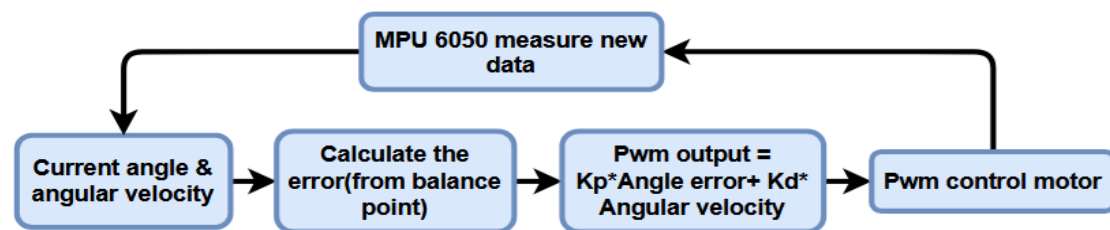


Figure 18. Operating Principle of Balance Loop Control

Given that the purpose of the balance loop is to maintain the vehicle's equilibrium, there is no manual input for a target angle in the balance loop; instead, a predetermined mechanical 0 degree serves as the target value. Then, as depicted in(Figure 18), the actual instantaneous angle and angular velocity are read by the MPU 6050 and compared with the target value to determine the errors in angle and angular velocity. From a balancing perspective, the vehicle needs to be stabilized at the mechanical 0 degree, hence the actual angle of the vehicle is considered a quantity that requires control. Since angular velocity is the derivative of the angle, the error in angular velocity allows for the determination of the control signal's derivative component. Subsequently, the instantaneous angle error is multiplied by a proportional parameter K_p , and the instantaneous angular velocity error is multiplied by a differential parameter K_d . The sum of these two products yields the angular velocity value of the control signal. This value is then decoded into a PWM signal, input into the motors, and enters the next cycle.

Why Use PD Control for Balance Loop

The balance control of a balancing vehicle necessitates the use of PD control, primarily because of the rapid response advantage offered by PD control. For the act of maintaining balance, it is imperative that the motor's response is sufficiently swift to ensure stability. Moreover, the inclusion of differential control is justified by the scenario where, as the angle approaches the mechanical zero

degree, the P controller ceases to operate, presuming the task has been completed. However, this overlooks a prediction for the subsequent moment, specifically that there still exists angular velocity in the motor when the angle is at the mechanical zero degree, typically caused by the vehicle's inertia. In such instances, the addition of differential control is essential as it enables the prediction of deviations in the next moment.

Why Do Not Use Integration Control for Balance Loop

The rationale for not employing integral control within the balance loop stems from the observation that the system exhibits negligible steady-state error when the proportional coefficient is substantial. The upright control of a balancing vehicle necessitates rapidity, resulting in a significantly large proportional control constant, K_p . Integral control is intended to eliminate steady-state error, which has minimal impact on upright control and, thus, is deemed unnecessary. Moreover, considering the practical aspects of balance performance, it is not imperative for the vehicle to be precisely at the mechanical midpoint, indicating that static balance is not a requirement. Instead, the vehicle's dynamic equilibrium is regarded as more congruent with real-world conditions. Dynamic balance also enables the vehicle to maintain stability without oscillating when subjected to external forces. Consequently, integral control has not been incorporated into the vehicle's balance loop.

2.6 Program and code design

Based on the previous feasibility analysis, concept design, and hardware selections, the core functionalities that need to be implemented through coding can be divided into the following three major sections as shown in the figure.

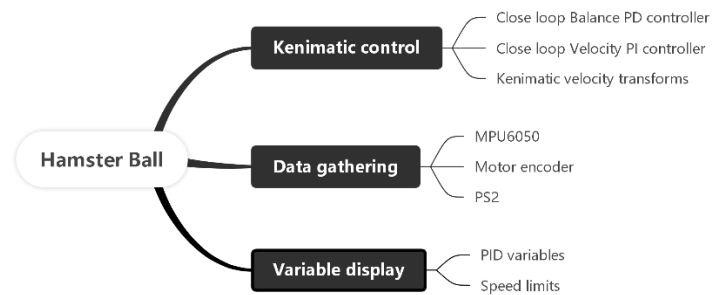


Figure 19. Design sectors for 'Hamster Ball'

Thus, the code design for this system can be started by realizing each function blocks

2.6.1 Kinematic Control Algorithms

The control system for this robot incorporates two Proportional-Integral-Derivative (PID) controllers to accommodate the self-balance motion, response for velocity control inputs, and the kinematic analysis for mapping the velocities between the X and Y axis and three omnidirectional wheels, for the control inputs and MPU inputs are all in X and Y axis. Thus, the objectives can be concrete as needing of function for balance PD response for roll and pitch motions, and velocity PI for input response of PS2 controller inputs in the X and Y axis, forward Kinematic analysis, and inverse Kinematic analysis.

The control functions implemented are as follows.

```

void Kinematic_Analysis(float Vx,float Vy,float Vz){
    pwm_a = kv*(Vy+2*Vz);
    pwm_b = kv*(-Vx * 0.5 * pow(3, 0.5) - Vy * 0.5 + 2*Vz);
    pwm_c = kv*(Vx * 0.5 * pow(3, 0.5) - Vy * 0.5 + 2*Vz);
}
  
```

Listing 1. Inverse kinematic

This function represents the inverse Kinematic analysis which calculates the required velocities (pwm_a, pwm_b, pwm_c) for each wheel based on the desired velocity vector of the robot (Vx,Vy,Vz). This function is fundamental in

translating the high-level motion commands into specific wheel speeds that adhere to the kinematic constraints of the robot.

```
void Forward_Kinematics(float Va,float Vb,float Vc){  
    Velocity_y=(Va*2-Vc-Vb)/kv/3;  
    Velocity_x=(Vc-Vb)/(sqrt(3)*kv);  
    Velocity_z=(Va+Vb+Vc)/kv/3;  
}
```

Listing 2. Forward kinematic

Similar to the inverse Kinematic function, this function processes the individual wheel velocities to determine the overall velocity and direction of the robot (Velocity_y, Velocity_x, Velocity_z). This is essential for predicting the robot's movement based on the current wheel speeds.

```
int balance_Roll(float Angle,float Gyro){  
    float Bias;  
    int balance;  
    Bias=Angle;  
    balance=Balance_Kp_Roll *Bias+Gyro*Balance_Kd_Roll/100;  
    return balance;  
}
```

Listing 3. Close loop PD control in Roll direction

Based on the analysis of balance PD control, this function calculates the necessary adjustments to the motor speeds to maintain balance, utilizing the current tilt angle and angular velocity in the Roll direction as feedback. Similarly for the balance PD loop in Pitch direction.

```
int balance_Pitch(float Angle,float Gyro){  
    float Bias;  
    int balance;  
    Bias=Angle;
```

```

    balance=Balance_Kp_Pitch *Bias+Gyro*Balance_Kd_Pitch;

    return balance;
}

```

Listing 4. Close loop PD control in Pitch direction

```

int velocity_X(float velocity, float target){
    static float err_sumx;

    float err = target - velocity;

    float p_out = err * Velocity_Kp_X;

    err_sumx += err;

    float i_out = err_sumx * Velocity_Ki_X;

    float out = p_out + i_out;

    return (int) out;
}

int velocity_Y(float velocity, float target){
    static float err_sumy;

    float err = target - velocity;

    float p_out = err * Velocity_Kp_Y;

    err_sumy += err;

    float i_out = err_sumy * Velocity_Ki_Y;

    float out = p_out + i_out;

    return (int) out;
}

```

Listing 5. Close loop PI control in X and Y direction

based on the analysis of Velocity PI control, to adjust the robot's speed smoothly to match the desired velocity, factoring in both the current velocity and the cumulative error (the integral of the velocity error over time, “err_sumx” and “err_sumy”), whereas discrete representative of integration. The loop for X and Y directions are similar in structure.

2.6.2 Sensor Reading

The control system is designed as a closed-loop mechanism allowing the robot to achieve stability and responsiveness in dynamic control. This needs feedback from the system. The sensor readings are crucial for feedback control systems.

MPU6050

Communicated using I2C (Inter-Integrated Circuit) protocol and utilizing the MPU6050 library online, which can read the yaw pitch roll angle from the built-in DMP (Digital Motion Processor) on the MPU module, while also be able to obtain the angular speed. Which are critical for balanced PD control. Based on the given example code in the library, the acquired variables can be obtained by following codes snippets.

```
mpu.dmpGetCurrentFIFOPacket(fifoBuffer);  
mpu.dmpGetQuaternion(&q, fifoBuffer);  
mpu.dmpGetGravity(&gravity, &q);  
mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);  
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
```

Listing 6. Lines of reading MPU DMP

“ypr” is an array to store the yaw pitch roll angle and “gx”, “gy”, “gz” are the angular velocity.

Alternative methods, such as quaternion and Kalman filter algorithms, offer other means of angle computation, but the MPU6050 library was selected for its direct applicability and ease of integration.

Motor Encoder

Motor encoders are utilized to provide velocity feedback, essential for dynamic speed adjustments. The motor encoders used in this project are the hall sensor type which has two hall sensors working as a pair on each motor which can generate square waves while the magnet plate mounted at the back of the

motor rotor spin. Thus, the speed can be obtained by calculating the times the hall sensor has been triggered in a constant period. The counter function can be triggered using the built-in input interrupt feature of the Arduino mega board.

In order to obtain a more precise reading without overloading the interrupt, the code had been designed as x2 frequency doubling. Frequency doubling is a technique used to enhance the resolution of the encoder without increasing the physical number of sensors or changing the encoder's design. This technique takes advantage of the fact that each transition (either from high to low or low to high) in the signal from a Hall sensor can be counted as an individual event, effectively doubling the resolution of the encoder. Practical code snippets are as follows (taking input interrupt for motor a as an example).

```
void READ_ENCODER_a() {  
  if (digitalRead(ENCODER_a) == LOW) {    //if triggered by falling edge  
    if (digitalRead(DIRECTION_a) == LOW)    EncoderData_a--; //define the direction by  
voltage level other encoder line  
    else    EncoderData_a++;  
  }  
  else {    //if triggered by rising edge  
    if (digitalRead(DIRECTION_a) == LOW)    EncoderData_a++; // define the direction by  
voltage level other encoder line  
    else    EncoderData_a--;  
  }  
}
```

Listing 7. Encoder reading method

PS2 Controller

User interaction is facilitated through a PS2 controller, which allows for the real-time acquisition of user commands, by using the PS2X_lib.h library and referencing the given example, the function enable reading control inputs can be as follows.

```

    ps2x.read_gamepad(false, 0); //read controller and set large motor to spin at 'vibrate'
    speed
    PS2_LX=ps2x.Analog(PSS_LX);
    PS2_LY=ps2x.Analog(PSS_LY);
    PS2_RX=ps2x.Analog(PSS_RX);
    PS2_RY=ps2x.Analog(PSS_RY);
    readcommand();
    Target_x = (PS2_LY - 128);
    Target_y = (PS2_LX - 127);
    Target_z = -(PS2_RX - 127);

```

Listing 8. PS2 Reading

The “readcommand()” function contains input reading of different buttons on the controller and mapping to the corresponding adjustment to PID variables and speed limits. The format is as follows.

```

void readcommand(){
    if(ps2x.Button(PSB_TRIANGLE)){
        Velocity_Kp+=0.3;
        delay(timedelay);
        OLEDdisplay();
    }
    else if(ps2x.Button("other button names")){
        //specific functions//
        delay(timedelay);
        OLEDdisplay();
    } .....
}

```

Listing 9. Botton detection & variable adjustment

Variable Display

The 0.96-inch OLED module display provides real-time feedback on PID

parameter readings and the velocity limit. By including the Adafruit library (Adafruit_SSD1306.h), the display can be achieved by following code snippets.

```
void OLEDdisplay(){
    display.clearDisplay();//clear register
    display.setCursor(30,0); //define print start location
    display.println("Velocity PID");//print header
    display.setCursor(22,12);
    display.print("P:");
    display.print(Velocity_Kp, 1);//print variable
    .....
    display.display();//display register
    display.clearDisplay();
}
```

Listing 10. Variable display

2.6.3 Coding Implementation

What has been mentioned in the previous section are all function blocks which need to be linked together. Moreover, implementing the control system into practical Arduino codes necessitated a focus on operational speed within each cycle to enhance PID control performance. Recognizing the importance of a high control rate, several optimization techniques were adopted.

Selective OLED Refresh. To minimize unnecessary processing, the OLED display is only refreshed upon changes in variables, reducing the operational load on the microcontroller.

IIC Transmission Speed Increase. Accelerating the data transmission rate to fast mode (400000), ensuring timely and accurate sensor data processing.

Additionally, PID control greatly benefits from maintaining a constant control rate. This consistency is crucial because the effectiveness of PID algorithms

depends on precise timing for calculating the error terms and applying the correct adjustments. When the control rate is constant, the PID controller can accurately predict system behaviour and respond with the appropriate corrections, ensuring smoother and more stable control. This constant time control can be realised by the library “MsTimer2.h” which enables the global time interrupt of the MCU.

Thus, the structural design of the whole control system is as follows.

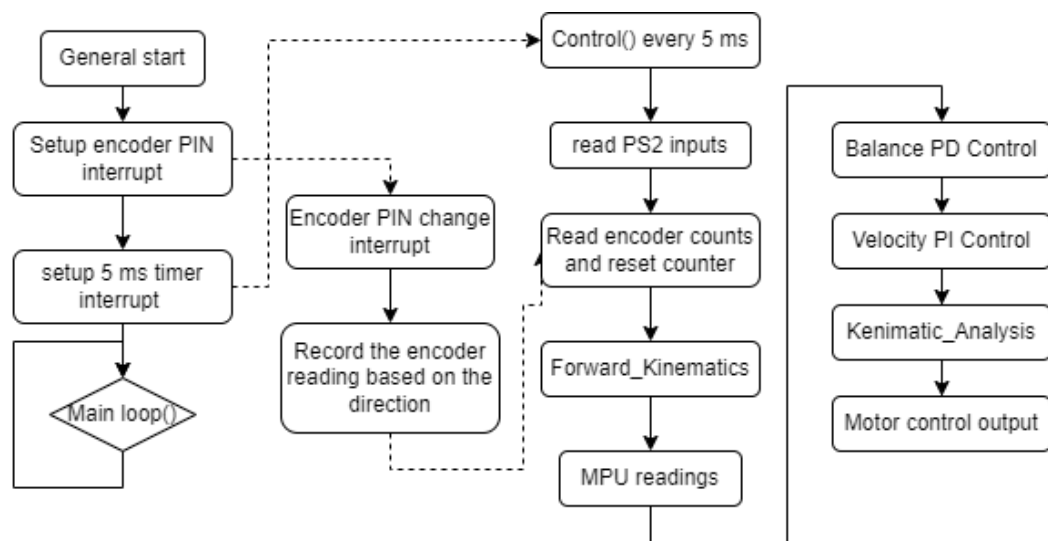


Figure 20. General structure of control system

Results

3.1 General performance

The research resulted in an advanced system with remote control capabilities and full omnidirectional movement. Advanced autonomous stability elements were incorporated to enhance operating performance significantly. The team significantly improved its capacity to construct and enhance complicated robotic mechanisms through acquiring skills in mechanical design, advanced 3D printing processes, programming proficiency, and PID controller optimisation. This progress highlights the project's achievement in fulfilling its goals and adds value to the wider realm of robotics and automated control systems.

3.2 Kinematics Analysis

Through detailed calculation and derivation, the mathematical relationships between the speeds of the three wheels around the motor and the vehicle's forward, backward, lateral, and rotational velocities are established. This establishes a foundation for further development of motor control code through inverse kinematics calculations based on existing correlations. Additionally, real-time feedback of each motor's speed can influence the vehicle's motion state through forward kinematics calculations, optimizing the response to the next control command and even stabilizing the vehicle's posture. The theoretical validation and calculations in this section provide assurance for subsequent development.

3.3 Mechanical Structure Design

3.3.1 Testing Platform

The development and utilization of the testing platform played a crucial role in

enhancing code optimization efficiency and hardware debugging convenience. Following the principle of rapid hardware iteration and synchronized updates with control code adaptation, the testing platform was developed and utilized. It was designed to stay as close as possible to the final design conditions while facilitating the initial validation of conceptual designs. Simultaneously, it provided necessary platform and trial-and-error opportunities for subsequent development and upgrades, ensuring the swift progression of the project.

3.3.2 Final Product

The general mechanical design of final product is successful.

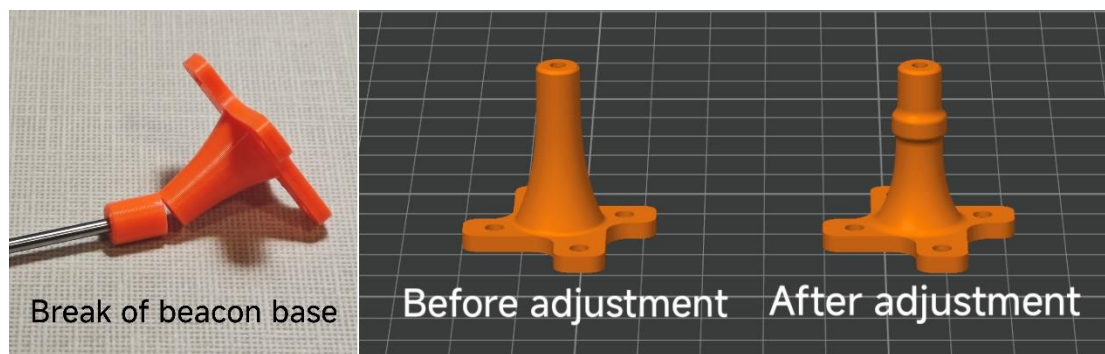


Figure 21. Comparison of beacon bases before and after the adjustment

Through thoughtful design, the process of 3D printing maximally avoided overhang printing (as shown in Figure 13), thereby ensuring print accuracy and component strength. Although there were instances of printing failures and component fractures (Beacon base shown in Figure 21) during testing, continuous adjustments and testing led to the successful creation of the "hamster ball." These processes ensured the strength of the structure, with no critical component damage occurring during testing.

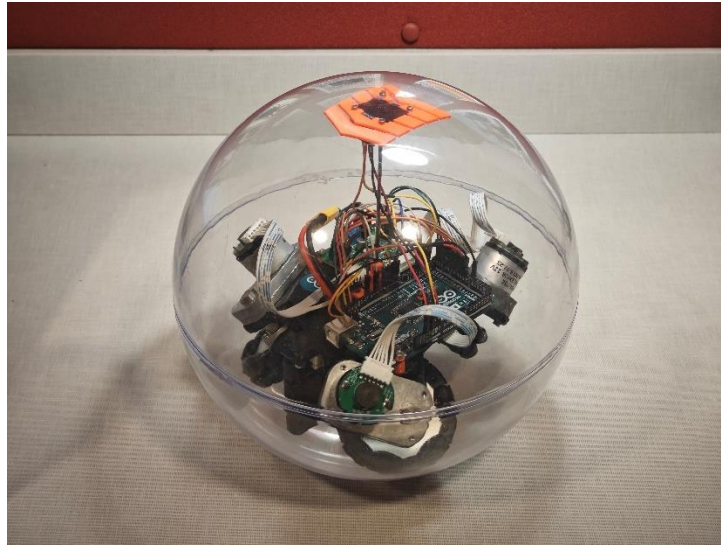
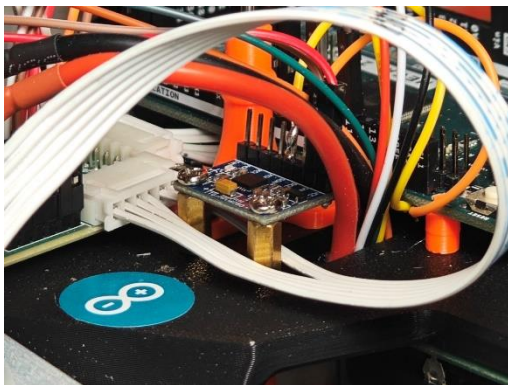


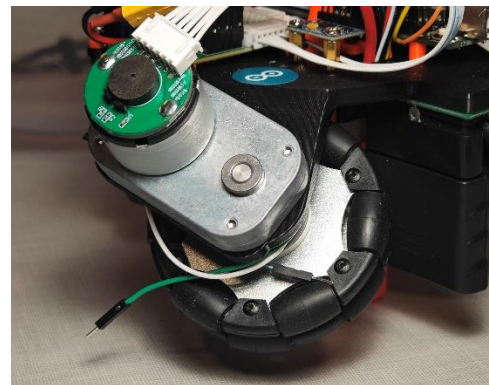
Figure 22. Self-balancing (while power off)

Through a rational distribution of the center of gravity, the vehicle achieved effective self-balancing (without PID), as illustrated in (Figure 22) , which plays a crucial role in the vehicle's balance performance. Additionally, due to the design of the beacon, the vehicle is very easy to control (the vehicle's direction and visual posture can be easily distinguished).

But there are also some issues that **need to be addressed for improvement.**



a) Wires entangling the wheel



b) The wire is drawn into the wheel

Figure 23. Machanical design where need to improve

Due to insufficient consideration of the wiring during the 3D modeling process, interference with the mechanical structure occurred in the final assembly of the product. However, since the wiring was easily adjustable, it did not significantly affect the final product (Figure 23 a). At the same time, because the wiring on

the vehicle was almost entirely exposed and not well constrained, the use of long wires posed a risk of entangling with the wheels during operation (Figure 23 b), potentially causing damage to the vehicle.

Due to the selection of a smooth acrylic material for the spherical shell, even if many designs are made to improve the driving efficiency, there is still a certain skid between the wheels and the ball shell when applying high acceleration or running on the ground with high friction. There are three ways to solve this problem.

- a. Limiting the vehicle's acceleration through programming. However, this could make the vehicle hard to control, resulting in slower operational responses and longer braking distances.
- b. Detecting slippage through programming. By integrating angular velocity, it is possible to detect whether the vehicle is slipping and make corrections (a conceptual approach to slippage detection).
- c. Changing materials (most effective). Increasing the friction coefficient between the shell and the wheels would enhance the vehicle's adaptability to different environments, as well as improve its maximum acceleration and operational response.

3.4 Hardware and Circuit Design

In this project, the general design of the hardware was successful, ultimately achieving the control of the vehicle. The wide use of finished modules avoided many potential issues that could arise during the design and soldering of PCB boards. Finally, all hardware is successfully connected and driven. However, some challenges were inevitably encountered.

- During the early stages of development, it was not realized that pins 20 and 21 on the Arduino Mega board were also used for the I2C bus. Leading to the motor's PWM control lines being mistakenly connected to these pins. When I2C communication was initiated later, it resulted in communication

interference, causing the vehicle to lose control.

- Furthermore, the use of pin headers and sockets for some hardware connections are not entirely reliable, leading to instances of vehicle control loss during both the testing platform and final product development due to poor contact. To address this issue, further development will require the design and soldering of a PCB board to encapsulate the mentioned modules. This approach can reduce the risks of poor contact and wires entangling with the wheels, while also enhancing the appearance of the project.

3.5 Program and Code Design

After implementing the control code based on the design structure onto the Arduino board, several problems occurred which severely affected the operation and control of the robot. These problems are unable to be detected without practical tests, which cannot be avoided forehanded. Thus, interactions in coding are needed.

The problems can be divided into two different sectors similar to the sections mentioned at the beginning of the coding design.

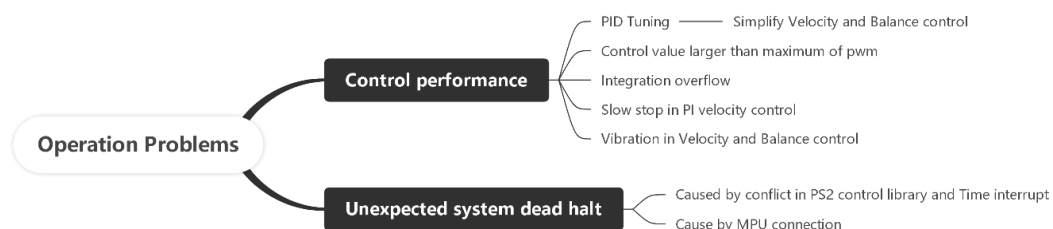


Figure 24. Operation problems

3.5.1 Control Performance

PID Tuning

PID tuning is the most crucial aspect which contributes to the general

performance of the control system, it is not only challenging but also time-consuming. Because of the limitation in experience in dynamic system modelling (need to consider friction, precise geometry centre, inertia), robotic simulations, and insufficient time, the tuning process cannot be done on software with clear analysis such as Ziegler-Nichols, Cohen-Coon to calculate each parameter.

It is also possible to manually adjust the parameters. Based on the previous analysis of this PD balance loop and PI velocity loop for this self-balance control system, two controllers are in a hierarchical relation where balance PD control is more fundamental (low-level), while velocity PI control is at a higher level, the parameters can be adjusted sequentially to achieve a desired system response. The process begins with only adjusting the proportional term. The P coefficient is increased until the system output begins to oscillate, finding a balance where the system is responsive but not overly sensitive to changes.

Once the proportional control leads to oscillation, the differential (D) control is introduced to add damping to the system. Damping helps to reduce the oscillation by counteracting the effect of sudden changes in error. It smooths out the system's response, making it more stable.

After Achieving the general magnitude of the balance PD parameters, the rest of the parameters of the PI controller can be obtained by adjusting based on the reading monitored by the Serial connection to the computer. After general tuning, more precise adjustments can be done through the PS2 controller dynamically.

The efficiency of this tuning process is significantly improved through the use of a PS2 controller, which allows for intuitive and real-time adjustments. Furthermore, adjustments to the velocity PI tuning can be easily made via serial

output, as detailed in the PID analysis section. This simplifies the tuning process by enabling direct and immediate feedback.

During the tuning process, it's observed that the PID parameters for balancing (PD for roll and pitch) and velocity (PI for x and y) exhibit similarities due to the system's symmetrical design. This observation suggests that a unified approach to parameter adjustment—treating roll and pitch, x and y parameters equivalently—can streamline the control functions, thereby simplifying the tuning process and potentially enhancing the overall system responsiveness and stability.

PWM Overflow

There is the possibility that the control outputs from the PD and PI controllers will be larger than the full occupation of PWM control. To address this issue, it's essential to add a limiting function. This function acts as a safeguard, "clipping" the combined control signal to the maximum threshold. This ensures that the signal remains within the permissible range, preventing potential overflow issues that could lead to erratic motor behaviour or system instability.

```
int Xianfu_Pwm(int pwm1)
{
    int out;
    int Amplitude=253; //Full 255
    out = pwm1;
    if(pwm1<-Amplitude) out=-Amplitude;
    if(pwm1>Amplitude) out=Amplitude;
    return out;
}
```

Listing 11. PWM limiting

Integration Overflow

When testing the robot, the integration component of the control algorithm,

represented by 'err_sum' is seen to windup to a very high value when the error is present over a long time, known as "integration windup". When the error decreases, this accumulated value causes the controller to overshoot the target, leading to instability and oscillations. Similar to the previously mentioned "PWM overflow" motors and their drivers can only accept control signals within certain limits. If the integral term is not limited, integration overflow can produce control signals outside the acceptable range, causing the actuator to saturate. Once in saturation, any further increase in the integral term won't result in a corresponding increase in motor speed or torque, potentially leading to a loss of control precision. Thus, the integration needs an upper limit where it cancels the steady state offset but does not saturate the system. The code snippets are as follows

```
if (out>1000) out =1000;  
if (out<-1000) out = -1000;
```

Listing 12. Integrate limitation

Slow Stop in PI Control

Another phenomenon which occurs is that the integration term decreases slowly when running for a long time which causes the system to have a poor response when trying to rapidly slowdown. The core of the issue lies in the integration term's inherent characteristic to accumulate error over time, enhancing system stability and counteracting steady-state errors under normal operating conditions. However, this beneficial accumulation becomes counterproductive when the system needs to quickly adapt to changes.

To address this issue and improve system responsiveness, a tailored strategy can be implemented where the integration term is manually forced to decrease at an accelerated rate once the input reaches a "0" state, signalling a need for an immediate stop or rapid slowdown. This manual adjustment ensures that the integration term quickly diminishes, thereby allowing the system to respond more effectively to sudden changes in desired state or control inputs.

```
if(target == 0){  
    if(err_sum>600)err_sum -= 500;  
    if(err_sum<-600)err_sum +=500;  
}
```

Listing 13. Fast decrease in integration

Input noise

As the figure shown in the detailed explanation of the PID (Proportional-Integral-Derivative) control system design, it is clear that the output signal of controllers has significant vibration. This problem can be caused by various reasons, such as noise introduced through sensor readings, distortions arising from trigonometric function conversions, anomalies generated during data type transitions, and the amplification of noise by the Proportional (P) control component.

This issue can be degraded by integrating a low-pass filter into the control system, specifically targeting the controller's input variables. A low-pass filter serves the crucial function of attenuating high-frequency noise while allowing the passage of low-frequency signals, which typically represent the mainstream of the system being controlled. By applying such a filter, the system can effectively smooth out rapid fluctuations in the input signal, thereby preventing the controller from reacting to spurious noise and instead focusing on genuine control tasks.

```
velocity *=0.7;  
velocity +=velocity_last*0.3 ;  
velocity_last = velocity;
```

Listing 14. Simple low-pass filter

3.5.2 Unexpected System Dead Halt

Despite resolving numerous issues within the control system, unexpected dead halts (DH) persisted, presenting a critical challenge. Detailed analysis revealed

that these interruptions were primarily caused by obstructions within the time interrupt processes. A notable contributing factor was identified as the Arduino's incompatibility with stacking time interrupts, a functionality that certain features within the PS2 controller library attempt to utilize. Moreover, DH incidents were also linked to unstable connections in the pin connectors utilized for I2C communication.

Because the control functions are closely linked with each other as shown in the flowchart in the previous section, thus by optimizing operation speed in each cycle, the functions in time interrupt can be moved to main loop function to eliminate the conflict between PS2 library and time interrupt (because it is hard to alternate the library), while achieving same performance. Additionally, the I2C bus for MPU had also been resoldered to achieve a more stable connection. These interventions significantly reduced the frequency of DH occurrences. However, recognizing the potential for residual issues, an additional safeguard was implemented in the form of a 'watchdog timer' (WDT) program. It acts as a safety mechanism that can reset the microcontroller within an Arduino board if the software becomes unresponsive or gets stuck in a loop. This capability is especially valuable in scenarios where continuous operation is critical, and manual resets are not feasible.

The WDT is essentially a timer that counts down from a predefined value. The software running on the Arduino must regularly reset this timer to prevent it from reaching zero. If the software fails to do so — typically due to being stuck in a loop or crashing — the WDT times out and automatically resets the microcontroller, thus helping to restore normal operation.

3.5.3 Final Representation

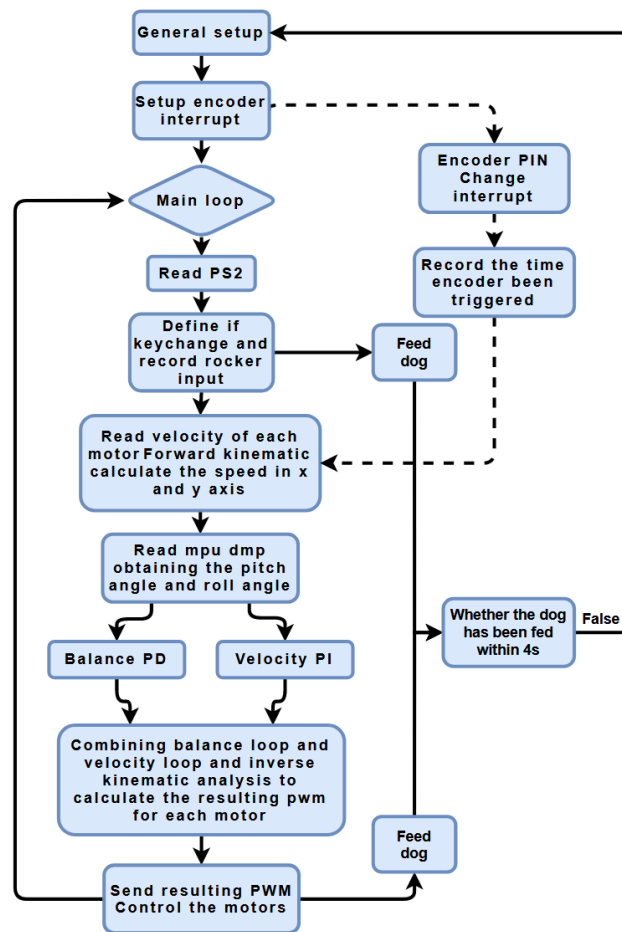


Figure 25. Final code structure

Discussion and Conclusion

4.1 Revisiting Previous Sections

We have thoroughly analyzed the application of PID control in our "Hamster Ball" autonomous vehicle. The results presented underscore the effectiveness of our design in ensuring stability and responsiveness.

4.2 Limitations Current and Future Work Applications

There is still much potential for improvement in the existing outcomes. Firstly, there is a slippage issue with the tyres and the sphere's wall due to the materials used [6]. Secondly, regarding intellect, the code can be upgraded to incorporate autonomous operation characteristics, provide obstacle avoidance functions, and enhance interface techniques. Finally, this idea could also be implemented in automobiles, intelligent assistants, and other applications.

4.3 Achievement and Contribution

Qi Cheng. The Designer/Developer, who focused on conceptualizing the vehicle's design, ensuring both functionality and aesthetic appeal.

Shaotong Hong. Role as a Developer involved tackling the complex coding challenges, particularly implementing the PID control for balance and motion.

Zijin Wu. Take on the role of Technical Writer and Implementer, focusing on documenting the process and ensuring the design meets the project's specifications.

Yunqi Wang. The Project Manager, who oversaw the project's progress, ensuring the team worked efficiently and cohesively towards the goal.

Junyang Xiao. Similarly, as a Technical Writer and Implementer, ensured the project was well-documented and contributed to the implementation phase.

4.4 Reflection

When considering the project experience, the most important lessons go beyond just technical accomplishments. The team thoroughly explored the complexities of intricate circuit connections and the subtleties of PID control, becoming proficient in the art of PID.

Debugging and optimising code. The learning curve for comprehending and implementing PID principles was challenging but highly gratifying. The foundation of the project's success was primarily the steadfast support from each member. This project was not just an academic pursuit but a demonstration of the influence of collaboration, perseverance, and mutual regard.

The team developed a sophisticated project and gained a deeper understanding of engineering principles, strengthening our team bond in the process. The whole team values these shared triumphs and struggles as the most cherished experiences.

References

- [1] Kiam Heong Ang, G. Chong, and Yun Li, "PID control system analysis, design, and technology," *IEEE Transactions on Control Systems Technology*, vol. 13, no. 4, pp. 559–576, Jul. 2005, doi. 10.1109/tcst.2005.847331. [Online]. Available: <http://dx.doi.org/10.1109/tcst.2005.847331>
- [2] K.-L. Su, Y.-L. Liao, S.-P. Lin, and S.-F. Lin, "An Interactive Auto-recharging System for Mobile Robots," *International Journal of Automation and Smart Technology*, vol. 4, no. 1, pp. 43–53, Mar. 2014, doi. 10.5875/ausmt.v4i1.197. [Online]. Available: <http://dx.doi.org/10.5875/ausmt.v4i1.197>
- [3] VT. Nguyen, QC. Nguyen, DH. Phan, TL. Bui and H. Xiem. "Optimized PID Controller for Two-Wheeled Self-balancing Robot Based on Genetic Algorithm," *Intelligent Systems and Networks*, vol. 752, pp. 60-65, Aug. 2023, doi. https://doi.org/10.1007/978-981-99-4725-6_9. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-99-4725-6_9#citeas
- [4] S. A. Junoh, "Two-wheeled balancing robot controller designed using pid," 2015. [Online]. Available: <https://core.ac.uk/download/pdf/42956061.pdf>
- [5] Ian, "What are good strategies for tuning PID loops?" [Online]. Available: <https://robotics.stackexchange.com/questions/167/what-are-good-strategies-for-tuning-pid-loops>
- [6] T. D. Ngo, A. Kashani, G. Imbalzano, K. T. Q. Nguyen, and D. Hui, "Additive manufacturing (3D printing). A review of materials, methods, applications and challenges," *Composites Part B. Engineering*, vol. 143, pp. 172–196, Jun. 2018, doi.10.1016/j.compositesb.2018.02.012. [Online]. Available: <http://dx.doi.org/10.1016/j.compositesb.2018.02.012>

Appendix

A.1 Y2 project (ELEC222/ELEC273) - Contribution to project deliverables

Table 1. Contribution to project deliverables

	Member name	Deliverable(s)	Comments
1	Qi.Cheng	Circuit/ mechanical structure/ Code	Final vehicle structure design and manufacturing. Circuit design. Write remote control system.
2	Shaotong.Hong	Code/testing	Write the code of the car. Implement PID system and code structure
3	Yunqi.Wang	Bench/Kinematic analysis	Organize the development process, push forward task progress Do the kinematic analysis. Compile weekly logbook.
4	Zijin.Wu	Blog/poster/Code	Make blog of week 1 and 3. Poster text. Study and participate in PID system and testing.
5	Junyang.Xiao	Blog/poster/Mechanical structure	Make blog of week 2 and 4. Poster edition. Testing and design chassis prototype.

A.2 Y2 project (ELEC222/ELEC273) – *Role allocation (responsibility matrix)*

Table 2. Role allocation

	Member Name	Title(s)
1	Qi.Cheng	Designer/developer
2	Shaotong.Hong	Developer
3	Zijin.Wu	Technical writer/implementer/Maintenance
4	Junyang.Xiao	Technical writer/implementer/Maintenance
5	Yunqi.Wang	Project manager/Designer

Responsibility Matrix

KEY:

R – Responsible (accountable) for completion of task.

S – Supports task.

C – Requires communication about the task.

Table 3. Responsibility Matrix

Title	Project Activity				Deliverables			
	Requirements/Scope	Design	Implementing	Testing	Poster	Blog	Bench	Report
Project Manager	R	C	S	S	S	S	R	S/R
Designer	C	R	S	S	S	S	R	S

Implementer/ Developer	C	S	R	R	C	C	R	S
Technical Writer	C	S	S	S	R	R	R	R

A.3 Code

```

#include <MsTimer2.h>
#include <math.h>
#include <PS2X_lib.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_Sensor.h>
#include <avr/wdt.h>
#include "Wire.h"
#include "MPU6050_6Axis_MotionApps20.h"

Adafruit_SSD1306 display = Adafruit_SSD1306(128, 64, &Wire);
PS2X ps2x;
MPU6050 mpu;
Quaternion q;
VectorFloat gravity;
//Define
//function defines
#define USE_ACCEL_LIMIT 0
//mpu
#define INTERRUPT_PIN A3
//PS2
#define PS2_DAT 5 //DI
#define PS2_CMD 6 //DO

```

```

#define PS2_SEL      38 //CS
#define PS2_CLK      40 //CLK

//MotorA

#define ENCODER_a 3
#define DIRECTION_a 46
#define pwma 2
#define ain1 22
#define ain2 24

//MotorB

#define ENCODER_c 18
#define DIRECTION_c 34
#define pwmc 11
#define cin1 26
#define cin2 28

//MotorC

#define ENCODER_b 19
#define DIRECTION_b 36
#define pwmb 8
#define bin1 32
#define bin2 30

//-----

//Variables

//PS2

unsigned char PS2_LX,PS2_LY,PS2_RX,PS2_RY;

//PID

float Velocity_Kp = 8;
float Velocity_Ki = 0.2 ;
float Balance_Kp = 40;
float Balance_Kd = 2;

//Encoder

```

```

volatile long EncoderData_a ;
volatile long EncoderData_b ;
volatile long EncoderData_c ;
int EVelocity_a= 0;
int EVelocity_b= 0;
int EVelocity_c= 0;

//Speed after forward kinematic
int Velocity_x= 0;
int Velocity_y= 0;
int Velocity_z= 0;

//OUPUTPWM

float Target_x, Target_y, Target_z; //PS2 input
float Balance_Pwm_X, Velocity_Pwm_X, Balance_Pwm_Y,
Velocity_Pwm_Y, Velocity_Pwm_Z;
float Move_x, Move_y;
int pwm_a = 0;
int pwm_b = 0;
int pwm_c = 0;


float kv = 0.15;
int timedelay = 10;

//MPU
int16_t ax, ay, az, gx, gy, gz;
bool dmpReady = false;
uint8_t mpulntStatus;
uint8_t devStatus;
uint8_t fifoBuffer[64];
float ypr[3];
volatile bool mpulInterrupt = false;

//temper

```

```

int flag = 0;
int i2cStatus = 0;
int Display_Count = 0;

//-----

//Functions

void dmpDataReady() {
    mpulInterrupt = true;
}

void Kinematic_Analysis(float Vx,float Vy,float Vz){
    pwm_a  = kv*(Vy+2*Vz);
    pwm_b  = kv*(-Vx * 0.5 * pow(3, 0.5) - Vy * 0.5 + 2*Vz);
    pwm_c  = kv*(Vx * 0.5 * pow(3, 0.5) - Vy * 0.5 + 2*Vz);
}

void Forward_Kinematics(float Va,float Vb,float Vc){

    Velocity_y=(Va*2-Vc-Vb)/kv/3;
    Velocity_x=(Vc-Vb)/(sqrt(3)*kv);
    Velocity_z=(Va+Vb+Vc)/kv/3;

}

int balance_Roll(float Angle,float Gyro){
    float Bias;
    int balance;
    Bias=Angle;
    balance=Balance_Kp*Bias+Gyro*Balance_Kd/100;
    return balance;
}

int balance_Pitch(float Angle,float Gyro){
    float Bias;
    int balance;

```



```

    Bias=Angle;

    balance=Balance_Kp*Bias+Gyro*Balance_Kd/100;

    return balance;
}

int velocity_X(float velocity, float target){
    static float err_sumx ,velocity_last_x;

    velocity *=0.7;

    velocity +=velocity_last_x*0.3;

    velocity_last_x = velocity;

    float err = target - velocity;

    float p_out = err * Velocity_Kp;

    err_sumx += err;

    float i_out = err_sumx * Velocity_Ki;

    float out = p_out + i_out;

    // Serial.print(out);

    // Serial.print(" ");

    if(target == 0){
        if(err_sum>600)err_sum -= 500;

        if(err_sum<-600)err_sum +=500;

    }

    if (out>1000) out =1000;

    if (out<-1000) out = -1000;

    return (int) out;

}

int velocity_Y(float velocity, float target){
    static float err_sumy ,velocity_last_y;

    velocity *=0.7;

    velocity +=velocity_last_y*0.3 ;

    velocity_last_y = velocity;

    float err = target - velocity;

```

```

float p_out = err * Velocity_Kp;
err_sumy += err;
float i_out = err_sumy * Velocity_Ki;
float out = p_out + i_out;
if(target == 0){
    if(err_sum>600)err_sum -= 500;
    if(err_sum<-600)err_sum +=500;
}
if (out>1000) out =1000;
if (out<-1000) out = -1000;
return (int) out;
}

void OLEDdisplay(){
    display.clearDisplay();
    display.setCursor(30,0);
    display.println("Velocity PID");
    display.setCursor(22,12);
    display.print("P:");
    display.print(Velocity_Kp, 1);
    display.setCursor(60,12);
    display.print(" I:");
    display.println(Velocity_Ki, 1);
    display.setCursor(32,27);
    display.println("Balance PID");
    display.setCursor(22,37);
    display.print("P:");
    display.print(Balance_Kp, 1);
    display.setCursor(60,37);
    display.print(" D:");
    display.println(Balance_Kd, 1);

```

```

display.setCursor(12,52);
display.print("Max Velocity: ");
display.print(kv);
display.display();
display.clearDisplay();

}

void readcommand(){
  if(ps2x.Button(PSB_TRIANGLE)){
    Velocity_Kp+=0.3;
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
  }
  else if(ps2x.Button(PSB_CROSS)){
    Velocity_Kp-=0.3;
    if(Velocity_Kp<=0){Velocity_Kp=0;}
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
  }
  else if(ps2x.Button(PSB_CIRCLE)){
    Velocity_Ki+=0.01;
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
  }
  else if(ps2x.Button(PSB_SQUARE)){
    Velocity_Ki-=0.01;
    if(Velocity_Ki<=0){Velocity_Ki=0;}

```

```

    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
}

else if(ps2x.Button(PSB_PAD_UP)){
    Balance_Kp+=0.4;
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
}

else if(ps2x.Button(PSB_PAD_DOWN)){
    Balance_Kp-=0.4;
    if(Balance_Kp<=0){Balance_Kp=0;}
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
}

else if(ps2x.Button(PSB_PAD_RIGHT)){
    Balance_Kd+=0.05;
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
}

else if(ps2x.Button(PSB_PAD_LEFT)){
    Balance_Kd-=0.05;
    if(Balance_Kd<=0){Balance_Kd=0;}
    delay(timedelay);
    OLEDdisplay();
    wdt_reset();
}

```

```

}
else if(ps2x.Button(PSB_START)){
  delay(3000);
}
else if(ps2x.Button(PSB_L1)){
  kv+=0.003;
  if(kv>=0.20){kv=0.20;}
  delay(timedelay);
  OLEDdisplay();
  wdt_reset();
}
else if (ps2x.Button(PSB_L2)){
  kv-=0.003;
  if(kv<=0){kv=0;}
  delay(timedelay);
  OLEDdisplay();
  wdt_reset();
}

}

```

```

void loop() {
  ps2x.read_gamepad(false, 0); //read controller and set large motor to
  spin at 'vibrate' speed
  PS2_LX=ps2x.Analog(PSS_LX);
  PS2_LY=ps2x.Analog(PSS_LY);
  PS2_RX=ps2x.Analog(PSS_RX);
  PS2_RY=ps2x.Analog(PSS_RY);
  readcommand();
}

```

```

Target_x = (PS2_LY - 128);
Target_y = (PS2_LX - 127);
Target_z = -(PS2_RX - 127);

```

```

EVelocity_a = EncoderData_a; EncoderData_a = 0; //Read encoder
readings and reset counter

```

```

EVelocity_b = EncoderData_b; EncoderData_b = 0;
EVelocity_c = EncoderData_c; EncoderData_c = 0;
Forward_Kinematics(EVelocity_a,EVelocity_b,EVelocity_c);

```

```

//mpu
mpu.dmpGetCurrentFIFOPacket(fifoBuffer);
mpu.dmpGetQuaternion(&q, fifoBuffer);
mpu.dmpGetGravity(&gravity, &q);
mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

```

```

//Balance loop
Balance_Pwm_X=balance_Pitch(ypr[2]*180/M_PI, gx); //Pitch direction
Balance_Pwm_Y=balance_Roll(-ypr[1]*180/M_PI, -gy); //Roll direction

```

```

Velocity_Pwm_X = velocity_X(Velocity_x, Target_x);
Velocity_Pwm_Y = velocity_Y(Velocity_y, Target_y);

```

```

Move_x = Balance_Pwm_X+Velocity_Pwm_X;
Move_y = Balance_Pwm_Y+Velocity_Pwm_Y;
Kinematic_Analysis(Move_x,Move_y,Target_z);

```

```

// Write motor control signal

if(pwm_a>0){
    digitalWrite(ain1,1);
    digitalWrite(ain2,0);}
else{
    digitalWrite(ain1,0);
    digitalWrite(ain2,1);}
analogWrite(pwma,abs(Xianfu_Pwm(pwm_a)));
if(pwm_b>0){
    digitalWrite(bin1,1);
    digitalWrite(bin2,0);}
else{
    digitalWrite(bin1,0);
    digitalWrite(bin2,1);}
analogWrite(pwmb,abs(Xianfu_Pwm(pwm_b)));
if(pwm_c>0){
    digitalWrite(cin1,1);
    digitalWrite(cin2,0);}
else{
    digitalWrite(cin1,0);
    digitalWrite(cin2,1);}
analogWrite(pwmc,abs(Xianfu_Pwm(pwm_c)));
wdt_reset();

}

void setup() {
    char error;

```

```

    error=ps2x.config_gamepad(PS2_CLK, PS2_CMD, PS2_SEL,
PS2_DAT, false, false); //ps2 init
    if(error)Serial.println("PS2 fail");
    Serial.begin(115200);

    Wire.begin();
    Wire.setClock(400000);
    // pinMode(INTERRUPT_PIN, INPUT);

    Serial.println(F("Initializing DMP..."));
    devStatus = mpu.dmpInitialize();

    if(devStatus == 0){
        mpu.CalibrateAccel(6);
        mpu.CalibrateGyro(6);
        mpu.PrintActiveOffsets();
        // turn on the DMP, now that it's ready
        Serial.println(F("Enabling DMP..."));
        mpu.setDMPEnabled(true);
        delay(10);

        Serial.print(F("Enabling interrupt detection (Arduino external interrupt
    "));
        Serial.print(digitalPinToInterrupt(INTERRUPT_PIN));
        Serial.println(F(")..."));
        attachInterrupt(digitalPinToInterrupt(INTERRUPT_PIN),
dmpDataReady, RISING);

        Serial.println(F("DMP ready! Waiting for first interrupt..."));
    }

```



```
mpu.setXGyroOffset(-290);  
mpu.setYGyroOffset(-26);  
mpu.setZGyroOffset(56);  
mpu.setZAccelOffset(1788);
```

```
pinMode(ain1, OUTPUT);  
pinMode(ain2, OUTPUT);  
pinMode(bin1, OUTPUT);  
pinMode(bin2, OUTPUT);  
pinMode(cin1, OUTPUT);  
pinMode(cin2, OUTPUT);
```

```
pinMode(pwma, OUTPUT);  
pinMode(pwmb, OUTPUT);  
pinMode(pwmc, OUTPUT);
```

```
pinMode(ENCODER_a, INPUT);  
pinMode(DIRECTION_a, INPUT);  
pinMode(ENCODER_b, INPUT);  
pinMode(DIRECTION_b, INPUT);  
pinMode(ENCODER_c, INPUT);  
pinMode(DIRECTION_c, INPUT);
```

```
delay(50);           //wait for init
```

```
if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Address  
0x3C for 128x32  
    Serial.println(F("SSD1306 allocation failed"));
```

```

    for (;;)
        ; // Don't proceed, loop forever
    }
    display.display();
    delay(500); // Pause
    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.clearDisplay();

    attachInterrupt(digitalPinToInterrupt(ENCODER_a) ,
    READ_ENCODER_a, CHANGE);
    attachInterrupt(digitalPinToInterrupt(ENCODER_b) ,
    READ_ENCODER_b, CHANGE);
    attachInterrupt(digitalPinToInterrupt(ENCODER_c) ,
    READ_ENCODER_c, CHANGE);

    wdt_enable(WDTO_4S);
}

void READ_ENCODER_a() {
    if (digitalRead(ENCODER_a) == LOW) { //if triggered by falling edge
        if (digitalRead(DIRECTION_a) == LOW)    EncoderData_a--; //define
the direction by voltage level other encoder line
        else    EncoderData_a++;
    }
    else { //if triggered by rising edge
        if (digitalRead(DIRECTION_a) == LOW)    EncoderData_a++; //
define the direction by voltage level other encoder line
        else    EncoderData_a--;
    }
}

```

```
}  
}
```

```
void READ_ENCODER_b() {  
    if (digitalRead(ENCODER_b) == LOW) {  
        if (digitalRead(DIRECTION_b) == LOW)    EncoderData_b--;  
        else    EncoderData_b++;  
    }  
    else {  
        if (digitalRead(DIRECTION_b) == LOW)    EncoderData_b++;  
        else    EncoderData_b--;  
    }  
}
```

```
void READ_ENCODER_c() {  
    if (digitalRead(ENCODER_c) == LOW) {  
        if (digitalRead(DIRECTION_c) == LOW)    EncoderData_c--;  
        else    EncoderData_c++;  
    }  
    else {  
        if (digitalRead(DIRECTION_c) == LOW)    EncoderData_c++;  
        else    EncoderData_c--;  
    }  
}
```

```
int Xianfu_Pwm(int pwm1)  
{  
    int out;  
    int Amplitude=253;    //Full 255
```

```
out = pwm1;  
if(pwm1<-Amplitude) out=-Amplitude;  
if(pwm1>Amplitude) out=Amplitude;  
return out;  
}
```